SWE580 Term Project

Tool Granularity in Personal Knowledge Retrieval

Project definition for comparing coarse-grained and fine-grained LLM tool interfaces

The Problem

You're building a system where an AI assistant helps users search through personal notes. How should you design the tools?

Option 1: Few powerful tools	**Option 2: Many simple tools**
One search tool accepts query, tags, dates, and other parameters.	Separate tools handle content, tags, dates, paths, titles, and links.

This project investigates which approach helps the AI succeed. You'll build both, test them on the same tasks, and analyze which performs better and why.

Research Question

Configuration	Tools	Interface
A	4	Coarse-grained tools with rich parameters
B	9	Fine-grained tools with simple parameters

How does tool granularity affect the effectiveness of LLM-driven personal knowledge retrieval?

You will implement and compare:

- **Configuration A:** 4 coarse-grained tools (rich parameters)
- **Configuration B:** 9 fine-grained tools (simple parameters)

Both use the same search backend and vault. Only the tool interface differs.

Test Environment

Standardized vault: 100 markdown notes across 5 categories (research, projects, meetings, daily journals, reference)

Each note has:

- Content (markdown with headers, lists, formatting)
- Metadata (tags, creation timestamp, modification timestamp)
- Links to other notes (wiki-style `[[links]]`)
- File path (organized in folders)

Key convention: Notes have no `title` field in frontmatter. Title is derived from filename (e.g., `Transformers.md` → title is "Transformers"). This ensures `[[Transformers]]` unambiguously refers to `Transformers.md`.

Test Queries

25 standardized queries across 6 categories:

1. **Simple Lookup** - "Get the note titled Transformers"
2. **Tag Search** - "Show me all my research notes"
3. **Temporal** - "Find notes from January 15, 2026"
4. **Content Search** - "Find research notes discussing attention mechanisms in depth"
5. **Multi-Faceted** - "Find research notes about transformers from January 2026"
6. **Graph-Based** - "What notes link to Transformers?"

Each query has ground truth answers (correct note paths).

Evaluation

The LLM receives your tool descriptions and decides which tools to call. The evaluator measures:

Metrics:

- Success rate (% queries returning correct notes)
- Tool call patterns (which tools, how many calls)
- Token efficiency (total tokens per query)
- Latency (response time)

Evaluation is at note-level resolution: Success means retrieving exactly the correct set of note paths.

Implementation Options

You may implement this project in one of two ways:

Option 1: MCP Servers Build two MCP servers using the Model Context Protocol. Test with Claude Desktop, VS Code MCP extensions, or any MCP-compatible client.

Option 2: Direct API Function Calling Implement tools as Python functions and evaluate via LLM APIs (Gemini, OpenAI, or Claude). Use provided evaluation scripts.

Both options must:

- Implement the same tool behaviors (4 tools vs 9 tools)
- Use the same Whoosh search backend
- Evaluate on the same 25 queries
- Collect the same metrics

Choose based on:

- Budget (Gemini API has free tier, Claude Desktop requires subscription)
- Learning goals (MCP teaches the protocol, API is simpler)
- Available resources

Document your choice and any limitations in your report.

Configuration A: 4 Coarse-Grained Tools

1. search_notes Search by content, tags, and/or date range. All parameters optional.

2. get_note Retrieve specific note by path or title.

3. get_related_notes Find notes linked to/from a specific note. Returns both outgoing and incoming links.

4. get_vault_overview Get vault statistics and recent activity.

Configuration B: 9 Fine-Grained Tools

Search:

7. search_by_content - Text query only
8. search_by_tags - Tags only (AND logic)
9. search_by_date - Date range only

Retrieval: 4. get_note_by_path - Exact path match 5. get_note_by_title - Title match

Links: 6. get_outgoing_links - Notes this note links to 7. get_incoming_links - Backlinks to this note

Stats: 8. get_vault_stats - Vault statistics 9. get_recent_notes - Recently modified notes

Technical Requirements

Technology Stack:

- Python 3.10+
- Whoosh search library (provided)
- MCP SDK (if using MCP path) or LLM API client (if using API path)

Constraints:

- Max 10 results per search
- Stateless tools (no session management)
- Synchronous operations
- Must use provided Whoosh backend

Critical Implementations:

- Single-day date queries must expand to full 24-hour range
- Links resolve via title = filename convention
- All tools must follow standardized return/error formats

See Technical Appendix for detailed formats and code examples.

Deliverables

1. Implementation

If using MCP:

- Two MCP servers (config_a/ and config_b/)
- Shared search backend
- README with setup instructions

If using API:

- Two Python modules with tool functions
- Evaluation script
- README with setup instructions

Both must include:

- requirements.txt
- Shared Whoosh search backend

2. Evaluation Results

- Automated evaluation on all 25 queries for both configs
- Raw data (JSON files with detailed metrics)

- Analysis of which queries succeeded/failed and why

3. Final Report

- Design: Configuration specifications
- Methodology: Evaluation setup, LLM used, metrics
- Results: Comparative metrics, statistical analysis
- Discussion: Which performed better? Why? Trade-offs? Recommendations
- Conclusion: Summary, limitations, future work

Grading

- **Implementation (30%):** Config A working (20%), Config B working (20%)
- **Evaluation (35%):** Automated evaluation complete and thorough
- **Analysis (30%):** Clear comparison (10%), Insightful analysis (5%), Justified recommendations (5%)
- **Documentation (5%):** Code quality & README (3%), Report clarity (2%)

Extra Credit (up to 10%):

- Implement note creation tools and test with synthesis queries (+5%)
- Test with multiple LLMs and compare results (+5%)